

DIETI UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II
DIPARTIMENTO DI INGEGNERIA ELETTRICA E DELLE TECNOLOGIE DELL'INFORMAZIONE

BASI DI DATI I

- RECAP su SQL
- Assertion

A.A. 2022/2023

Prof. Silvio Barra

STRUTTURA DI UN'INTERROGAZIONE

SELECT <*elenco attributi e funzioni*>

FROM <*elenco delle tabelle o viste*>

[**WHERE** <*condizioni*>]

[**GROUP BY** <*attributo o attributi di raggruppamento*>]

[**HAVING** <*condizione di raggruppamento*>]

[**ORDER BY** <*elenco attributi*>]



NELLA CLAUSOLA WHERE

- WHERE attributo op.rel. Valore
- WHERE attributo op.rel. Attributo
- WHERE attributo IS [NOT] NULL
- WHERE attributo IN (val1, val2, …)
- WHERE attributo BETWEEN val1 AND val2
- WHERE attributo LIKE *'pattern'*

- WHERE salario < 30000 (<>, =, <, <=, >,>=)
- WHERE M.salario > E.salario
- WHERE salario IS NOT NULL
- WHERE sesso IN ('M', 'F')
- WHERE salario BETWEEN 300 AND 400
- WHERE nome LIKE '_Mario%



DOVE ANNIDARE LE QUERY

SELECT <*elenco attributi e funzioni*>	NO
FROM <*elenco delle tabelle o viste*>	*SI*
[**WHERE** <*condizioni*>]	SI
[**GROUP BY** <*attributo o attributi di raggruppamento*>]	NO
[**HAVING** <*condizione di raggruppamento*>]	SI
[**ORDER BY** <*elenco attributi*>]	NO



RAGIONARE SULLA QUERY

- Le query possono essere di diverse complessità
- In tutti i casi, la richiesta è chiara e ci è molto semplice andare ad identificare ciò di cui abbiamo bisogno all'interno dello schema logico
- Risulta però complicato andare a ragionare su COME costruire la query che restituisca effettivamente l'output richiesto
- È SEMPRE importante andare a ragionare bene sulla richiesta, ed eventualmente trovare un altro modo per riscriverla, in modo che risulti più semplice andare a trasformarla in una query.



ESEMPIO

- Schema logico

  STUDENTE (MATRICOLA, NOME COGNOME)

  ESAME(MATRICOLA, CORSO)

- Query

  Gli studenti che hanno sostenuto almeno tutti gli esami sostenuti dallo studente con matricola 100

PROVIAMO A RISCRIVERLA?



ESEMPIO

- Schema logico

  STUDENTE (MATRICOLA, NOME COGNOME)

  ESAME(MATRICOLA, CORSO)

- Query

  Gli studenti che hanno sostenuto almeno tutti gli esami sostenuti dallo studente con matricola 100

  Trovo tutti gli studenti per cui NON ESISTA un esame che hanno sostenuto e che lo studente 100 non ha sostenuto

  L'insieme degli esami sostenuti dallo studente 100 e non sostenuti dallo studente x è vuoto



ESEMPIO

```
SELECT *
FROM Studenti AS S
WHERE NOT EXIST
        (SELECT E.Corso
         FROM Esami AS E
        WHERE E.Matricola ='100' AND E.Corso NOT IN
                                        (SELECT E2.Corso
                                        FROM Esami AS E2
                                        WHERE E2.Matricola = S.Matricola))
```



ESEMPIO

```
SELECT *
FROM Studenti AS S
WHERE NOT EXISTS
        (SELECT E.Corso
         FROM Esami AS E
        WHERE E.Matricola ='100' AND E.Corso NOT IN
```

```
(SELECT E2.Corso
FROM Esami AS E2
WHERE E2.Matricola = S.Matricola))
```

Insieme dei corsi sostenuti dallo studente S.Matricola



ESEMPIO

Insieme dei corsi sostenuti da 100, ma non sostenuti dallo studente S.Matricola



```
SELECT *
FROM Studenti AS S
WHERE NOT EXISTS
        (SELECT E.Corso
         FROM Esami AS E
        WHERE E.Matricola ='100' AND E.Corso NOT IN
                                        (SELECT E2.Corso
                                        FROM Esami AS E2
                                        WHERE E2.Matricola = S.Matricola))
```

ESEMPIO

```
SELECT *
FROM Studenti AS S
WHERE NOT EXISTS
        (SELECT E.Corso
         FROM Esami AS E
        WHERE E.Matricola ='100' AND E.Corso NOT IN
                                        (SELECT E2.Corso
                                        FROM Esami AS E2
                                        WHERE E2.Matricola = S.Matricola))
```

Insieme degli studenti S per cui l'nsieme degli esami sostenuti da 100, ma non sostenuti da S, è vuoto.

ATTENZIONE ALLA VISIBILITÀ DEI NOMI

```
SELECT *
FROM Studenti AS S
WHERE NOT EXIST
        (SELECT E.Corso
         FROM Esami AS E
        WHERE E.Matricola ='100' AND E.Corso NOT IN
                                        (SELECT E2.Corso
                                        FROM Esami AS E2
                                        WHERE E2.Matricola = S.Matricola))
```



VINCOLI E ASSERTION



VINCOLI

- I vincoli analizzati fino ad ora sono supportati dal linguaggio di definizione dei dati in quanto si manifestano nella maggior parte dei casi e nella maggior parte delle applicazioni per DB
  - Vincoli sui valori null e valori predefiniti
  - Vincoli sulle tuple (CHECK)
  - Vincoli sui domini degli attributi
  - Vincoli di chiave
  - Vincoli di integrità referenziale
- Esistono altri tipi di vincoli i quali, siccome non comuni a tutte le basi di dati, non fanno parte del DDL e devono essere specificati in maniera differente
  - Sono chiamati **vincoli di integrità semantica**



VINCOLI DI BASE

- CONSTRAINT <nome_vincolo> <vincolo>
- <vincolo>: = CHECK <espressione booleana>

  UNIQUE (<lista_attributi>)

  PRIMARY KEY (<lista attributi>)

  FOREIGN KEY (<lista_attributi_FK>) REFERENCES <nome_tabella>(lista_att_PK)

  [ON DELETE <azione>]

  [ON UPDATE <azione>]
- <azione>:= NO ACTION | CASCADE | SET DEFAULT | SET NULL



VINCOLI DI INTEGRITÀ SEMANTICA

- «Un impiegato non può guadagnare più del suo supervisore»
- «Ad un progetto non possono lavorare più di *n* impiegati»
- «Un impiegato non può lavorare più di 48 ore settimanali»

- Questa tipologia di vincoli può essere specificata
  - Sia all'interno dei programmi applicativi
  - Sia tramite un linguaggio di specifica dei vincoli

- SI possono utilizzare costrutti quali ASSERTION e TRIGGER



TIPOLOGIA DI VINCOLI

- In linea di massima i vincoli possono essere di 4 tipi
  - Table-level constraints
    - (PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK)
  - Column-level constraints
    - (PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, NOT NULL)
  - Schema-level constraints
    - Vincoli a livello di schema. Ogni volta che l'intero DB cambia stato, si controlla se un vincolo è violato



VINCOLI INTRARELAZIONALI E INTERRELAZIONALI

- La maggior parte dei vincoli sono intrarelazionali:
  - Per verificarne l'eventuale rispetto si deve andare a controllare esclusivamente la tabella in cui è avvenuta la modifica
    - PRIMARY KEY (vado a verificare che non esistano attributi uguali per la chiave primaria nella tabella dove ho effettuato la modifica)
    - UNIQUE (lo stesso)
    - NOT NULL (vado a verificare che il valore inserito per un attributo non sia null)
    - CHECK (Controllo che i valori dati ad uno o più attributi della tabella in cui è definito il vincolo, rispettino un determinato range/dominio)
- Il vincolo di integrità referenziale (FOREIGN KEY) è un vincolo *interrelazionale*, in quanto va a controllare che il valore definito nell'attributo (o negli attributi) esista nella tabella di riferimento.



VINCOLO DI CHECK

- È un costrutto molto potente
  - TEORICAMENTE Tutti i vincoli di tabella o di colonna (UNIQUE, PRIMARY KEY, FOREIGN KEY, …) possono essere espressi tramite un vincolo di CHECK.
    - Almeno in Postgres questo non funziona perché non è possibile creare subquery nei vincoli di check.
  - Far riferimento sia agli attributi della tabella in cui è definito, sia ad attributi di altre tabelle.
    - Anche questo in Postgres non è possibile.

```
create table impiegato(
matricola char(8) PRIMARY KEY,
dipartimento char(8),
check (matricola like 'c%' OR dipartimento in
       (SELECT nomedip
        FROM dipartimento)));
```

```
create table impiegato(
matricola char(8) PRIMARY KEY,
dipartimento char(8),
check(1>=(SELECT count(*)
          FROM impiegato I
          WHere matricola = I.matricola)
    )
   );
```

VINCOLI INTERRELAZIONALI

- Quando un vincolo di CHECK coinvolge due o più tabelle ci possono essere comportamenti indesiderati.
- Nell'esempio sotto, se inseriamo nella tabella DIPARTIMENTO più di 100 dipartimenti, il vincolo rimane soddisfatto, dato che il controllo viene fatto **SOLO** quando si inserisce la tupla all'interno della tabella IMPIEGATO

Cosa succede se vado a fare l'update di *nomedip* in DIPARTIMENTO?

```
create table impiegato(
matricola char(8) PRIMARY KEY,
dipartimento char(8),
check (matricola like 'c%' OR dipartimento in
        (SELECT nomedip
         FROM dipartimento)));
```

```
create table impiegato(
matricola char(8) PRIMARY KEY,
dipartimento char(8),
check((SELECT count(DISTINCT dipartimento)FROM impiegato)+
      (SELECT count(nome) FROM dipartimento)<100));
```

ASSERTION

- Per evitare situazioni spiacevoli e anomalie causate dai vincoli di check su più tabelle, si possono usare le **assertion**
  - Questi sono vincoli non collegati ad alcun attributo o tabella in particolare, ma appartengono direttamente allo schema di basi di dati (Schema-Level)

CREATE ASSERTION <nome vincolo> espressione booleana

```
CREATE ASSERTION <nome_assertion>
check (condizione)
```

```
1  CREATE ASSERTION ass1
2  check ((SELECT count(distinct nomedip)+10<100)));
```

Data output | Messages | Notifications

```
ERROR:  CREATE ASSERTION is not yet implemented
SQL state: 0A000
```

ASSERTION

CREATE ASSERTION <nome_vincolo> espressione booleana

[NOT] EXISTS (SELECT

<Cerco la violazione del vincolo>

)



ESEMPIO

ALBERO(CodAlbero, CodRadice)

NODE(CodNodo, Label, CodAlbero)

ARCO(CodArco, NodoSorgente, NodoTarget, Label, CodAlbero)

Vincoli

- *Sorgente e target di un albero sono diversi*



ESEMPIO

ALBERO(CodAlbero, CodRadice)

NODE(CodNodo, Label, CodAlbero)

ARCO(CodArco, NodoSorgente, NodoTarget, Label, CodAlbero)

Vincoli

- *Sorgente e target di un albero sono diversi (vincolo intrarelazionale, in particolare è un vincolo di ennupla) → non ho bisogno di un vincolo di asserzione, ma posso inserire un vincolo nella tabella.*



ESEMPIO

ALBERO(CodAlbero, CodRadice)

NODE(CodNodo, Label, CodAlbero)

ARCO(CodArco, NodoSorgente, NodoTarget, Label, CodAlbero)

Vincoli

1)Sorgente e target di un albero sono diversi (vincolo intrarelazionale, in particolare è un vincolo di ennupla) → non ho bisogno di un vincolo di asserzione, ma posso inserire un vincolo nella tabella.

ALTER TABLE ARCO

ADD CONSTRAINT C1

CHECK NodoSorgente <> Nodo Target



ESEMPIO

ALBERO(CodAlbero, CodRadice)

NODE(CodNodo, Label, CodAlbero)

ARCO(CodArco, NodoSorgente, NodoTarget, Label, CodAlbero)

Vincoli

2) L'identificativo della radice è un nodo dell'albero



ESEMPIO

ALBERO(CodAlbero, CodRadice)

NODE(CodNodo, Label, CodAlbero)

ARCO(CodArco, NodoSorgente, NodoTarget, Label, CodAlbero)

Vincoli

2) L'identificativo della radice è un nodo dell'albero (interrelazionale)

```
CREATE ASSERTION A1
CHECK NOT EXIST (SELECT  * FROM ALBERO AS T
                         WHERE T.CodRadice NOT IN
                         (SELECT N.CodNodo FROM NODO N
                         WHERE N.CodNodo= T.CodRadice )
```



ALTRI VINCOLI INTERRELAZIONALI

- Un nodo ha un solo padre
- Una radice non ha padri
- Tutti i nodi sono connessi



FINE

Per eventuali domande: (in ordine di preferenza personale)

- Ora.
- Chat di Teams
- Mail: silvio.barra@unina.it